Introduction to Computer Science and Programming

Labo 10 – C++ Exercises (Strings mainly)

Laboratory 10, December 16, 2025

Filippo Ricca

DIBRIS, University of Genova

Today…

Today's lab is dedicated to **exercises in C++**

- Exercise 1: Compute change (already assigned)
- Exercise 2: Tokenizer
- Exercise 3: Binary to decimal conversion
- …

All the program must be delivered on Aulaweb!

Exercise 1: Compute change

- Write a pseudocode that lets the user **insert a decimal amount representing a price in euros and euro cents**
  - E.g., 3.87
- The pseudocode will compute the **optimal change** for that amount using the best combination of coins (using the largest coins):
  - 2 euros, 1 euro, 50 cents, 20 cents, 10 cents, 5 cents, 2 cents, 1 cent
- The pseudocode must print the optimal change, one row per each type of coin
- Next, translate the pseudocode into C++

Example

- If we consider an amount of **3.87 euro**, our pseudocode (and then our C++ program) should print:

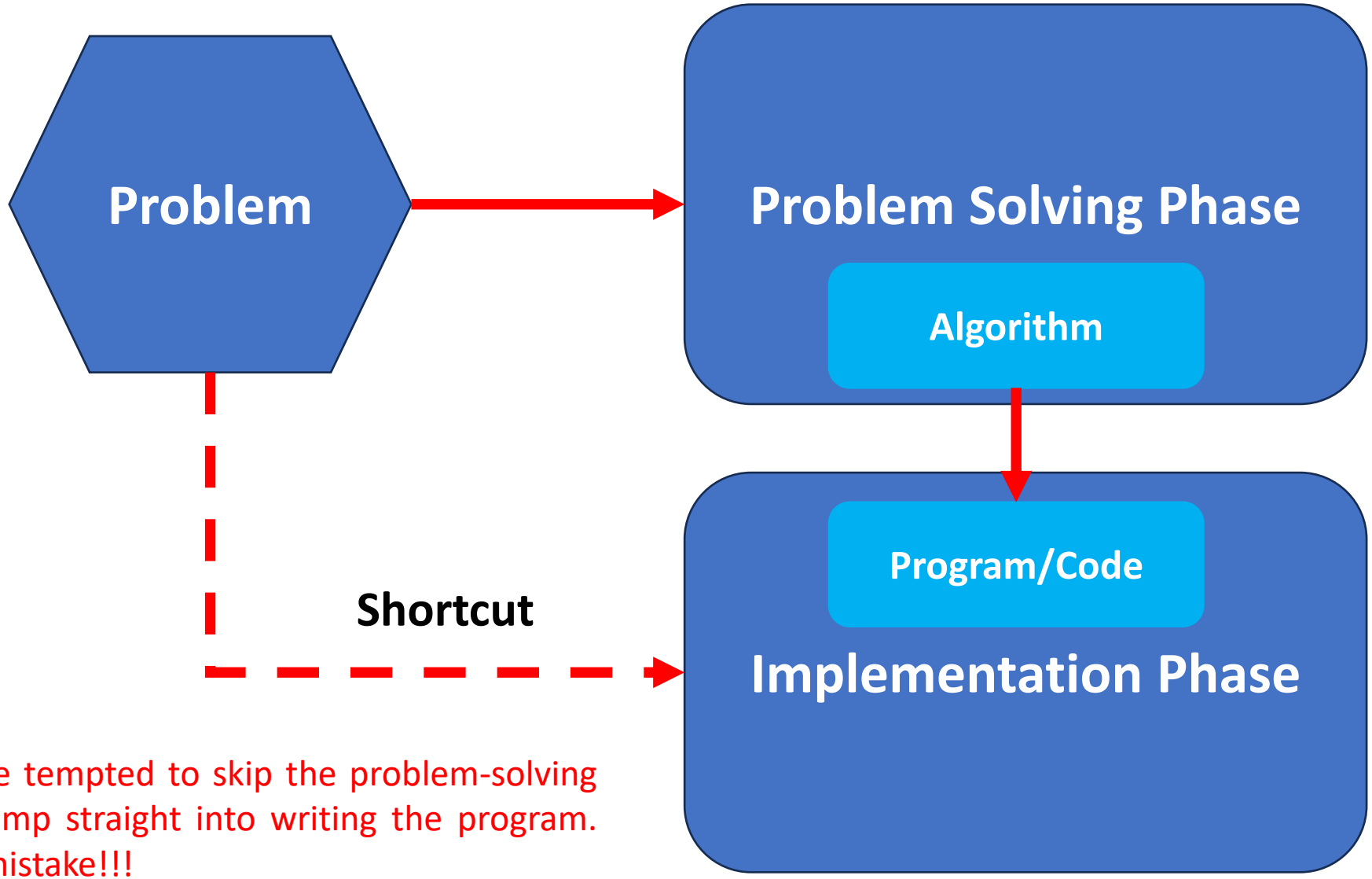
```
1 coin(s) of 2 euro(s)
1 coin(s) of 1 euro(s)
1 coin(s) of 50 cent(s)
1 coin(s) of 20 cent(s)
1 coin(s) of 10 cent(s)
1 coin(s) of 5 cent(s)
1 coin(s) of 2 cent(s)
```

3.87 euro

best combination of coins

(It means, at each step, using the largest coins possible)

Always avoid shortcuts!



Reasoning at level of Pseudocode!

One might be tempted to skip the problem-solving phase and jump straight into writing the program. This is a big mistake!!!

How do we arrive at the pseudocode?

How do we arrive at the pseudocode? Let's think about a possible solution

- **Step 1. 'prompt the user to insert a decimal amount representing a price'**
  - e.g. 3.87
- **Step 2. 'convert the decimal amount in cent'**
  - i.e. 387
  - Not so simple! **What if the user insert 3.8723? We have to round it**
- **Step 3. 'define Coin Types'**
  - 2 euros (200 cents), 1 euro (100 cents), 50 cents, 20 cents, 10 cents, 5 cents, 2 cents, 1 cent
- **Step 4: 'start with the Largest Coin'**
  - The program calculates how many 2-euro coins fit into 387 cents: 387 / 200 = 1 (1 coin used, 187 cents left)

Integer division

How do we arrive at the pseudocode?

- **Step 5: ‘move to the Next Coin’**
  It then calculates how many 1-euro coins fit into 187 cents:
  187 / 100 = 1 (1 coin used, 87 cents left)
- **Step 6: ‘continue with Smaller Coins’**
  - 50-cent coins: 87 / 50 = 1 (1 coin used, 37 cents left)
  - 20-cent coins: 37 / 20 = 1 (1 coin used, 17 cents left)
  - 10-cent coins: 17 / 10 = 1 (1 coin used, 7 cents left)
  - 5-cent coins: 7 / 5 = 1 (1 coin used, 2 cents left)
  - 2-cent coins: 2 / 2 = 1 (1 coin used, 0 cents left)

- **Step 7: ‘print the Results’**
  - 1 x 2-euro coin, 1 x 1-euro coin, 1 x 50-cent coin, 1 x 20-cent coin, 1 x 10-cent coin, 1 x 5-cent coin, 1 x 2-cent coin

From pseudocode to C++

```
• #include <iostream>
• using namespace std;
• int main() {
• // Step 1. ‘prompt the user to insert a decimal amount representing a price’
• // Step 2. ‘convert the decimal amount in cent’
• // Step 3. ‘define Coin Types’
• // Step 4: ‘start with the Largest Coin
• // Step 5: ‘move to the Next Coin’
• // Step 6: ‘continue with Smaller Coins’
• // Step 7: ‘print the Results’
• }
```

We have to copy in a C++ program the pseudocode steps

```
• #include <iostream>
• using namespace std;
• int main() {
• // Step 1. ‘prompt the user to insert a decimal amount representing a price’
    • double amount;
    • cout << "Enter an amount in euros (e.g., 12.34): ";
    • cin >> amount;
• // Step 2. ‘convert the decimal amount in cent’
• // Step 3. ‘define Coin Types’
• // Step 4: ‘start with the Largest Coin
• // Step 5: ‘move to the Next Coin’
• // Step 6: ‘continue with Smaller Coins’
• // Step 7: ‘print the Results’
• }
```

```
#include <iostream>
using namespace std;

int main() {
// Step 1. ‘prompt the user to insert a decimal amount representing a price’
    double amount;
    cout << "Enter an amount in euros (e.g., 12.34): ";
    cin >> amount;
// Step 2. ‘convert the decimal amount in cent’
    int totalCents = static_cast<int>(amount * 100 + 0.5); // Round to nearest cent
// Step 3. ‘define Coin Types’
// Step 4: ‘start with the Largest Coin
// Step 5: ‘move to the Next Coin’
// Step 6: ‘continue with Smaller Coins’
// Step 7: ‘print the Results’
}
```

It’s not that simple to convert the decimal amount in cent because a user could also enter more than two digits after the decimal point, for example 12.342 or 12.347.

In the first case, it would make sense to round down (1234), and in the second case, to round up (1235).

When you use static_cast<int> on a double, it performs a type conversion from double to int. This operation truncates (not rounds) the decimal part of the floating-point number, effectively rounding it down towards zero.

For example:

- **static_cast<int>(3.75) would result in 3**
- **static_cast<int>(-3.75) would result in -3**

```
#include <iostream>
using namespace std;

int main() {
// Step 1. ‘prompt the user to insert a decimal amount representing a price’
    double amount;
    cout << "Enter an amount in euros (e.g., 12.34): ";
    cin >> amount;
// Step 2. ‘convert the decimal amount in cent’
    int totalCents = static_cast<int>(amount * 100 + 0.5); // Round to nearest cent
// Step 3. ‘define Coin Types’
    int coins2Euro = 200;
    int coins1Euro = 100;
    int coins50Cent = 50;
    int coins20Cent = 20;
    int coins10Cent = 10;
    int coins5Cent = 5;
    int coins2Cent = 2;
    int coins1Cent = 1;
// Step 4: ‘start with the Largest Coin
// Step 5: ‘move to the Next Coin’
// Step 6: ‘continue with Smaller Coins’
// Step 7: ‘print the Results’
}
```

```
#include <iostream>
using namespace std;

int main() {
// Step 1. ‘prompt the user to insert a decimal amount representing a price’
    double amount;
    cout << "Enter an amount in euros (e.g., 12.34): ";
    cin >> amount;
// Step 2. ‘convert the decimal amount in cent’
    int totalCents = static_cast<int>(amount * 100 + 0.5); // Round to nearest cent
// Step 3. ‘define Coin Types’
    int coins2Euro = 200;
    int coins1Euro = 100;
    int coins50Cent = 50;
    int coins20Cent = 20;
    int coins10Cent = 10;
    int coins5Cent = 5;
    int coins2Cent = 2;
    int coins1Cent = 1;
// Step 4: ‘start with the Largest Coin
    int count2Euro = totalCents / coins2Euro;
    totalCents %= coins2Euro;
// Step 5: ‘move to the Next Coin’
// Step 6: ‘continue with Smaller Coins’
// Step 7: ‘print the Results’
}
```

% remainder or modulus operator

totalCents = totalCents % coins2Euro;

Ex. count2Euro = 387 / 200 = 1

totalCents = 387 % 200 = 187

```
#include <iostream>
using namespace std;

int main() {
// Step 1. ‘prompt the user to insert a decimal amount representing a price’
    double amount;
    cout << "Enter an amount in euros (e.g., 12.34): ";
    cin >> amount;
// Step 2. ‘convert the decimal amount in cent’
    int totalCents = static_cast<int>(amount * 100 + 0.5); // Round to nearest cent
// Step 3. ‘define Coin Types’
    int coins2Euro = 200;
    int coins1Euro = 100;
    int coins50Cent = 50;
    int coins20Cent = 20;
    int coins10Cent = 10;
    int coins5Cent = 5;
    int coins2Cent = 2;
    int coins1Cent = 1;
// Step 4: ‘start with the Largest Coin
    int count2Euro = totalCents / coins2Euro;
    totalCents %= coins2Euro;
// Step 5: ‘move to the Next Coin’
    int count1Euro = totalCents / coins1Euro;
    totalCents %= coins1Euro;
// Step 6: ‘continue with Smaller Coins’
// Step 7: ‘print the Results’
}
```

```cpp
#include <iostream>
using namespace std;

int main() {
// Step 1. ‘prompt the user to insert a decimal amount representing a price’
    double amount;
    cout << "Enter an amount in euros (e.g., 12.34): ";
    cin >> amount;
// Step 2. ‘convert the decimal amount in cent’
    int totalCents = static_cast<int>(amount * 100 + 0.5); // Round to nearest cent
// Step 3. ‘define Coin Types’
    int coins2Euro = 200; int coins1Euro = 100; int coins50Cent = 50;
    int coins20Cent = 20; int coins10Cent = 10; int coins5Cent = 5;
    int coins2Cent = 2; int coins1Cent = 1;
// Step 4: ‘start with the Largest Coin
    int count2Euro = totalCents / coins2Euro;
    totalCents %= coins2Euro;
// Step 5: ‘move to the Next Coin’
    int count1Euro = totalCents / coins1Euro;
    totalCents %= coins1Euro;
// Step 6: ‘continue with Smaller Coins’
    int count50Cent = totalCents / coins50Cent;
    totalCents %= coins50Cent;
    int count20Cent = totalCents / coins20Cent;
    totalCents %= coins20Cent;
    int count10Cent = totalCents / coins10Cent;
    totalCents %= coins10Cent;
    int count5Cent = totalCents / coins5Cent;
    totalCents %= coins5Cent;
    int count2Cent = totalCents / coins2Cent;
    totalCents %= coins2Cent;
    int count1Cent = totalCents / coins1Cent;
    totalCents %= coins1Cent;
// Step 7: ‘print the Results’
}
```

```
#include <iostream>
using namespace std;

int main() {
…
…
…
// Step 5: ‘move to the Next Coin’
    int count1Euro = totalCents / coins1Euro;
    totalCents %= coins1Euro;
// Step 6: ‘continue with Smaller Coins’
    int count50Cent = totalCents / coins50Cent;
    totalCents %= coins50Cent;
    int count20Cent = totalCents / coins20Cent;
    totalCents %= coins20Cent;
    int count10Cent = totalCents / coins10Cent;
    totalCents %= coins10Cent;
    int count5Cent = totalCents / coins5Cent;
    totalCents %= coins5Cent;
    int count2Cent = totalCents / coins2Cent;
    totalCents %= coins2Cent;
    int count1Cent = totalCents / coins1Cent;
    totalCents %= coins1Cent;
// Step 7: ‘print the Results’
    cout << "Optimal change:\n";
    if (count2Euro > 0) cout << count2Euro << " coin(s) of 2 euro(s)\n";
    if (count1Euro > 0) cout << count1Euro << " coin(s) of 1 euro(s)\n";
    if (count50Cent > 0) cout << count50Cent << " coin(s) of 50 cent(s)\n";
    if (count20Cent > 0) cout << count20Cent << " coin(s) of 20 cent(s)\n";
    if (count10Cent > 0) cout << count10Cent << " coin(s) of 10 cent(s)\n";
    if (count5Cent > 0) cout << count5Cent << " coin(s) of 5 cent(s)\n";
    if (count2Cent > 0) cout << count2Cent << " coin(s) of 2 cent(s)\n";
    if (count1Cent > 0) cout << count1Cent << " coin(s) of 1 cent(s)\n";

    return 0;
}
```

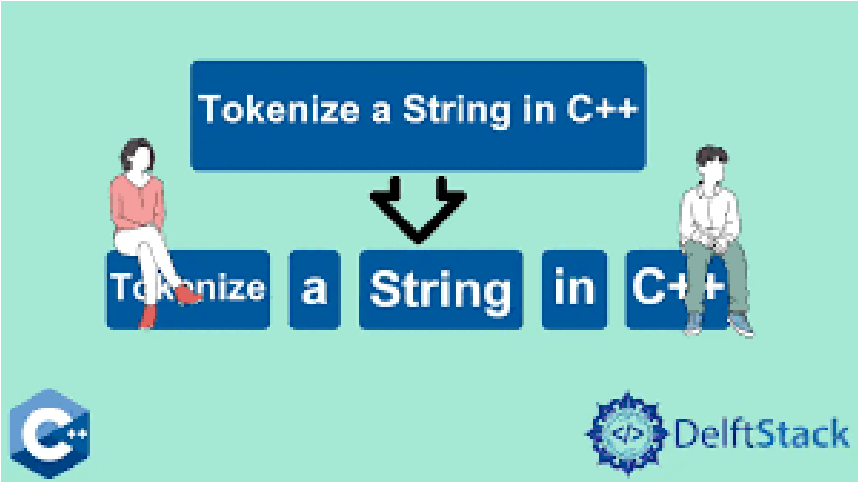
Exercise 2: Tokenizer

- Write a pseudocode that is able to **split a string into a number of sub-strings, one per line, providing a delimiter (i.e., one char)**

- The string and the delimiter must be inserted by the user

- For example:
  - **String:** How_do_you_do!
  - **Delimiter:** '_'
  - **Output:**
    - How
    - do
    - you
    - do!

- Next, translate the pseudocode into C++



Possible Pseudocode

USE: for (int i = 0; i < (int)str.size(); i++)

We have introduced 'for-loop' in lesson 3

```
1. Begin the program.
2. Display: "Enter the string you want to split."
3. Accept the user input and store it as `str`.
4. Display: "Enter the delimiter character."
5. Accept the user input and store it as `del`.
6. Initialize an empty string `temp` to store the current segment.

7. Loop through each character `ch` in `str`:
   a. If `ch` is not equal to `del`:
      i. Add `ch` to `temp`.
   b. Else:
      i. Print `temp` (current segment).
      ii. Reset `temp` to an empty string.

8. After the loop, check if `temp` is not empty:
   a. If it is not empty, print `temp` (last segment).

9. Display: "String splitting completed."
10. End the program.
```

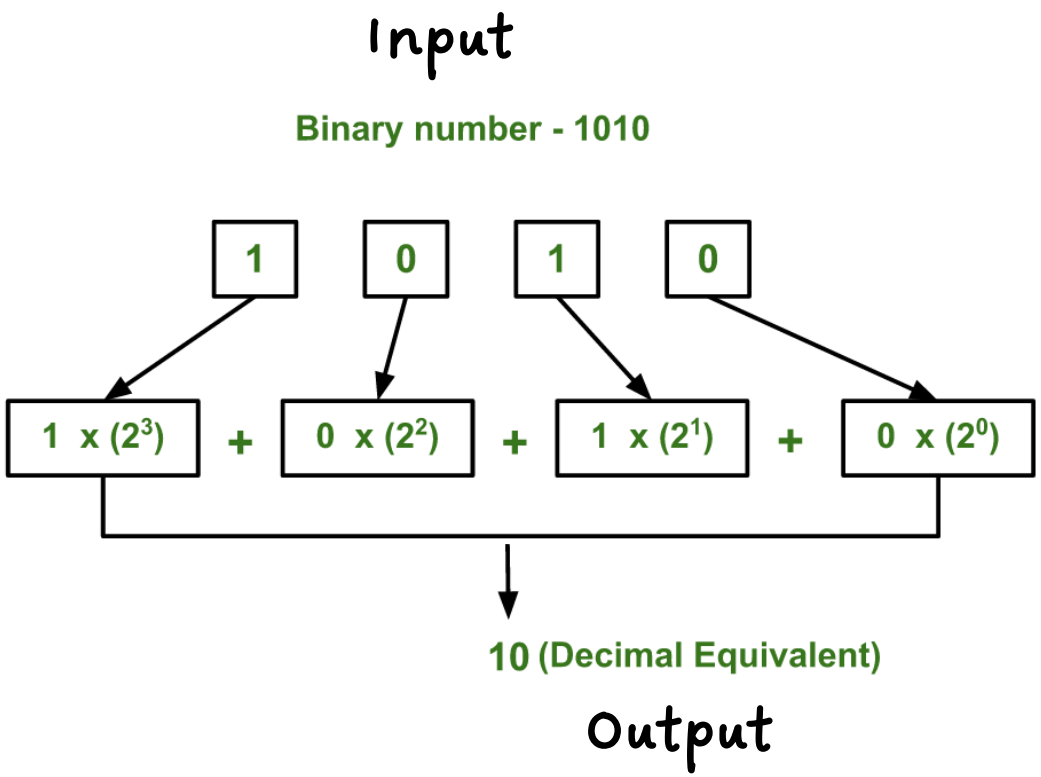
Transform it in C++

Now, it’s your turn

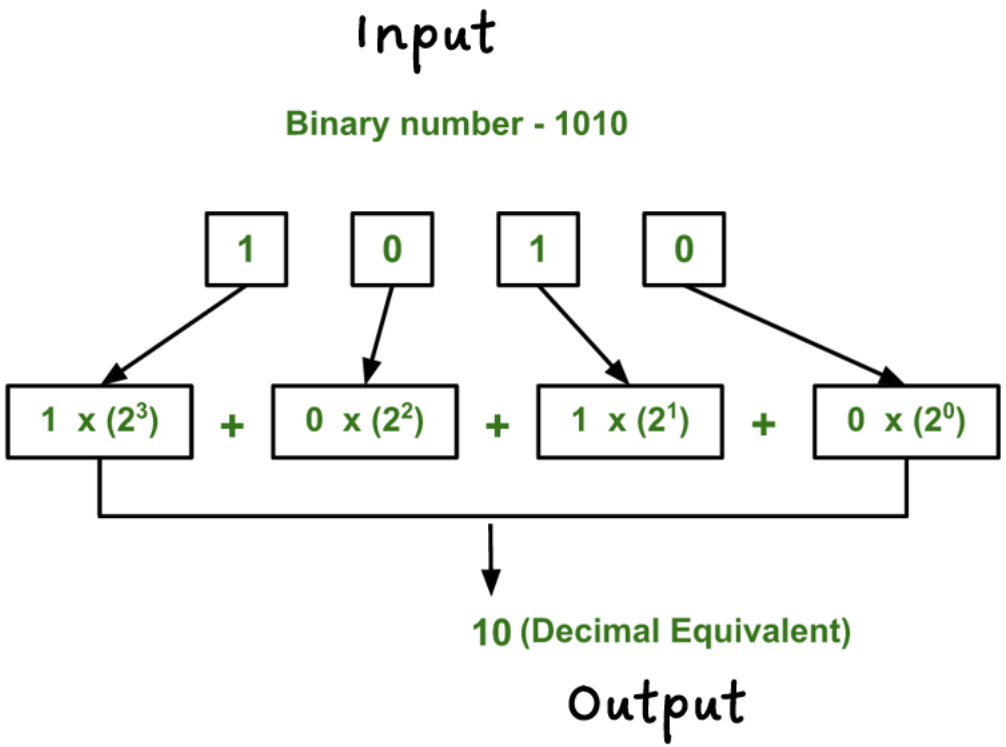
- **String:** How_do_you_do!
- **Delimiter:** ‘_’
- **Output:**
  - How
  - do
  - you
  - do!

Exercise 3: Binary To Decimal Conversion

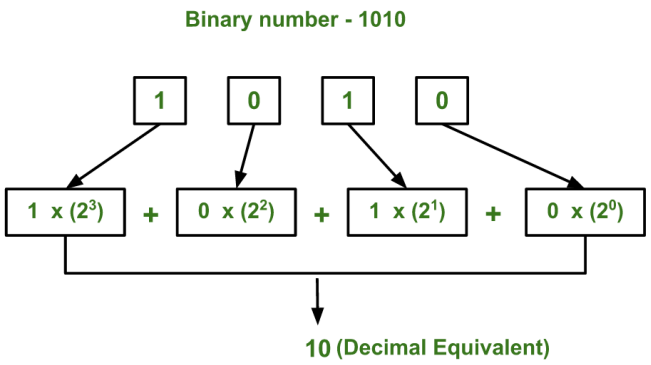
- The **binary number system** uses only two digits **0 and 1** to represent an integer and the **Decimal number system** uses ten digits **0 to 9** to represent a number
- You have to design a pseudocode first and then a C++ program for **Binary to Decimal conversion in C++**
- The program ask **an integer that correspond to a binary number** to the user and print out the correspondent Decimal number
- The diagram on the right explains how to convert (1010) to an equivalent decimal value (10)



Now, it's your turn



Possible Pseudocode



We have introduced 'while' in lesson 3

```
1. Begin the program
2. Display: "Enter the binary number to convert to decimal"
3. Accept the user input and store it in `num`

4. Initialize `dec_value` to 0 (to store the decimal result).
5. Initialize `base` to 1 (representing \(2^0\)).

6. While `num` is not 0:
   a. extract the last binary digit and put it in last_digit
   b. remove the last digit from num
   c. Update `dec_value` as `dec_value + (last_digit * base)`
   d. Update `base` as `base * 2` (move to the next binary place)

7. Display `dec_value` as the decimal equivalent of the binary number

8. End the program.
```

Exercise 4: The Caesar's code

- The Caesar cipher is a **shift cipher** since the ciphertext alphabet is derived from the plaintext alphabet by shifting each letter a certain number of spaces.
- For example, if we use a shift of ’19’, then we get the following pair of ciphertext and plaintext alphabets:
  - Plaintext: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
  - Ciphertext: T U V W X Y Z A B C D E F G H I J K L M N O P Q R S

- To **encipher a message, we perform a simple substitution** by looking up each of the message's letters in the top row and writing down the corresponding letter from the bottom row
- For example, the following is an encryption with the Caesar's code
  - THE FAULT, DEAR BRUTUS, LIES NOT IN OUR STARS BUT IN OURSELVES
  - MAX YTNEM, WXTK UKNMNL, EBXL GHM BG HNK LMTKL UNM BG HNKLXEOXL

Exercise 4: The Caesar's code

- Write a program to **encrypt and decrypt a string using Caesar cipher**.
- The program should:
  - Take a **string** as input.
  - Take a **shift value** as input for the Caesar cipher.
- The program should be able to:
  - **Encrypt** the string by shifting each character by the given shift value.
  - **Decrypt** (reverse-cypher) a string to recover the original text.
- Implement the functions using **C++ string**

Exercise 5: Converting User Input to Uppercase

- Write a C++ program that takes a string input from the user and converts it into uppercase letters. The program should:
  - Prompt the user to enter a string
  - Convert each character of the string to uppercase
  - Output the modified string

- Steps to Guide the Solution:
  - Use the toupper() function to convert characters to uppercase
  - Iterate through the string, applying toupper() to each character
  - Print the transformed string

Exercise 6: Palindrome Checker

```
Enter a word: level
level is a palindrome!
```

Create a C++ program that:

- Reads a string from the user.
- Checks whether the string is a **palindrome**
  - A **palindrome** is a word that reads the same **forward and backward**
- Prints:
  - "Yes" if the string is a palindrome.
  - "No" if the string is **not** a palindrome

```
Input: radar
Output: Yes

Input: hello
Output: No
```